

raylib-beginner-guide

Raylib Beginner's Guide

What is Raylib?

Raylib is a simple and easy-to-use library for learning game programming. It is written in C and provides bindings for C++. Raylib is perfect for beginners because:

- **Simple API:** Clean, straightforward functions that are easy to understand
- **No external dependencies:** Everything you need is included
- **Cross-platform:** Works on Windows, macOS, Linux, and even web browsers
- **Well-documented:** Extensive examples and documentation available
- **Active community:** Friendly community ready to help beginners

Raylib can help you create:

- 2D and 3D games
 - Interactive applications
 - Visualizations and simulations
 - Prototypes and proof-of-concepts
-

Prerequisites

Before installing Raylib, you should have:

1. **Basic C++ knowledge:** Understanding of variables, loops, functions, and basic syntax
 2. **A C++ compiler:** Different for each platform (we'll cover installation below)
 3. **An IDE or text editor:** Visual Studio, VS Code, or Xcode (We recommend VS and Xcode)
-

Installation Guides

Visual Studio (Windows)

First try `raylib_quickstart` provided by raylib. You can find it through the cloud link as follows:

<https://cloud.tsinghua.edu.cn/f/9d457c32c0ec444ca074/?dl=1>

which will download a zip file(raylib-quickstart-main2026.zip). You can unzip it and open the folder.

if you are able to connect to github, you may be able to successfully run the program by following the instructions in the README.md file.

If you are not able to connect to github, or you did not see the files compiled in the external folder, please try the following steps:

1. download a new zip file from the cloud link:
<https://cloud.tsinghua.edu.cn/f/a8bfca7f920a44e29f41/?dl=1>
which is raylib_master.zip.
2. copy raylib_master.zip to the external folder.
3. run the batch file again.
4. you should be able to see the files compiled in the external folder.

If the previous steps did not work and you are using visual studio 2022, you can download a complete folder which holds all the files you need to build and run the program.

<https://cloud.tsinghua.edu.cn/d/d04b040a523d49ed8e75/>

If the previous steps did not work, please wait until the tutorial session.

Xcode (macOS)

For the Xcode user, First try raylib_quickstart provided by raylib. You can find it through the cloud link as follows:

<https://cloud.tsinghua.edu.cn/f/9d457c32c0ec444ca074/?dl=1>

which will download a zip file(raylib-quickstart-main2026.zip). You can unzip it and open the folder.

You may be able to successfully run the program by following the instructions in the README.md file.

If the previous steps did not work, please try the following steps:

Step 1: Install Xcode

1. Open the **App Store** on your Mac
2. Search for "**Xcode**"
3. Click **Get** and then **Install**

4. Wait for the download and installation to complete (this is a large download, ~10+ GB)

Step 2: Install Xcode Command Line Tools

1. Open **Terminal** (Applications → Utilities → Terminal)
2. Run:

Code block

```
1  xcode-select --install
```

3. Click **Install** in the dialog that appears

Step 3: Install Raylib

You have two options:

Option A: Using Homebrew (Easiest)

1. If you don't have Homebrew, install it:

Code block

```
1  /bin/bash -c "$(curl -fsSL
    https://raw.githubusercontent.com/Homebrew/install/HEAD/install.sh)"
```

2. Install Raylib:

Code block

```
1  brew install raylib
```

Option B: Build from Source

Code block

```
1  git clone https://github.com/raysan5/raylib.git
2  cd raylib
3  mkdir build && cd build
4  cmake .. -DBUILD_SHARED_LIBS=ON
5  make
6  sudo make install
```

Step 4: Create a New Xcode Project

1. Open Xcode
2. Click **"Create a new Xcode project"** (or File → New → Project)
3. In the template selector, choose **macOS** → **Command Line Tool**
4. Click **Next**

Step 5: Configure the Project

1. Project Details:

- Product Name: `MyFirstRaylibGame`
- Organization: (your name or leave blank)
- Language: **C++**
- Click **Next**

2. Choose a location to save your project

3. Add Raylib to the project:

- In the Project Navigator (left sidebar), click on your project name
- Select your target under **TARGETS**
- Go to **Build Settings** tab
- Search for **"Header Search Paths"**
- Add the Raylib include path:
 - If using Homebrew on Apple Silicon: `/opt/homebrew/include`
 - If using Homebrew on Intel: `/usr/local/include`
 - If built from source: `/usr/local/include`

4. Add Library Search Paths:

- Search for **"Library Search Paths"** in Build Settings
- Add the Raylib library path:
 - Apple Silicon: `/opt/homebrew/lib`
 - Intel Mac: `/usr/local/lib`

5. Link the Raylib Library:

- Go to **Build Phases** tab
- Expand **"Link Binary With Libraries"**
- Click the + button
- Click **"Add Other..."** → **"Add Files..."**

- Navigate to the Raylib library:
 - Apple Silicon: `/opt/homebrew/lib/libraylib.dylib`
 - Intel: `/usr/local/lib/libraylib.dylib`
- Select the file and click **Open**

6. Add Required Frameworks:

- In the same "**Link Binary With Libraries**" section, click +
- Add the following frameworks (search for each and add):
 - `CoreVideo.framework`
 - `IOKit.framework`
 - `Cocoa.framework`
 - `GLUT.framework`
 - `OpenGL.framework`

Step 6: Write Your Code

1. In the Project Navigator, find `main.cpp`
2. Replace the contents with the sample code from "Your First Raylib Program" section

Step 7: Build and Run

1. Click the **Play button** (▶) in the top-left corner
2. Or press `Cmd+R`
3. Your Raylib window should appear!

Your First Raylib Program

Here's a simple program to test your Raylib installation. Create a file named `main.cpp` with this code:

Code block

```
1  #include "raylib.h"
2
3  int main() {
4      // Initialize the window
5      const int screenWidth = 800;
6      const int screenHeight = 450;
7
8      InitWindow(screenWidth, screenHeight, "My First Raylib Program");
```

```

9
10 // Set the game to run at 60 frames-per-second
11 SetTargetFPS(60);
12
13 // Main game loop
14 while (!WindowShouldClose()) { // Detect window close button or ESC key
15     // Update
16     // TODO: Add game logic here
17
18     // Draw
19     BeginDrawing();
20
21     ClearBackground(RAYWHITE);
22
23     DrawText("Congrats! You created your first window!",
24             190, 200, 20, LIGHTGRAY);
25
26     DrawCircle(screenWidth/2, screenHeight/2, 50, RED);
27
28     EndDrawing();
29 }
30
31 // De-initialize
32 CloseWindow();
33
34 return 0;
35 }

```

Understanding the Code

Function	What It Does
<code>InitWindow()</code>	Creates a window with specified width, height, and title
<code>SetTargetFPS()</code>	Limits the game to run at 60 frames per second
<code>WindowShouldClose()</code>	Returns true if the window should close (ESC key or close button)
<code>BeginDrawing()</code>	Starts the drawing phase
<code>ClearBackground()</code>	Fills the screen with a color
<code>DrawText()</code>	Draws text on the screen

<code>DrawCircle()</code>	Draws a circle
<code>EndDrawing()</code>	Ends the drawing phase and updates the screen
<code>CloseWindow()</code>	Closes the window and frees resources

Project Structure

A typical Raylib project structure:

Code block

```
1  MyRaylibProject/
2  |— src/
3  |   |— main.cpp
4  |   |— player.cpp
5  |   |— player.h
6  |   |— utils.cpp
7  |— include/
8  |   |— (third-party headers)
9  |— lib/
10 |   |— (third-party libraries)
11 |— assets/
12 |   |— images/
13 |   |— sounds/
14 |   |— fonts/
15 |— build/
16 |   |— (compiled output)
17 |— README.md
```

Learning Resources

Official Resources

- **Raylib Website:** <https://www.raylib.com/>
- **Raylib Examples:** <https://www.raylib.com/examples.html>
- **Raylib Cheatsheet:** <https://www.raylib.com/cheatsheet/cheatsheet.html>
- **Raylib GitHub:** <https://github.com/raysan5/raylib>

Community

- **Raylib Discord:** Join the official Discord for help and discussion
- **r/raylib Subreddit:** Community discussions and showcases

Tutorial Series

- **Raylib Tutorial Series:** Search YouTube for "raylib tutorial beginner"
- **Programming Knowledge:** Various channels offer step-by-step Raylib guides

Next Steps

1. Study the official examples
 2. Try modifying the sample code
 3. Create a simple game (like Pong or Snake)
 4. Experiment with different shapes, colors, and inputs
 5. Learn about textures, sounds, and game state management
-

Troubleshooting

Common Issues and Solutions

Issue: "raylib.h not found"

- Solution: Check that the include path is correctly set in your project settings

Issue: "undefined reference to..." linker errors

- Solution: Make sure the library is correctly linked in your project settings

Issue: Window doesn't open or crashes immediately

- Solution: Ensure `InitWindow()` is called before any other Raylib function

Issue: Black screen

- Solution: Make sure `BeginDrawing()` and `EndDrawing()` are properly paired
-

Part 2: Raylib Basics and Chess Game Tutorial

Now that you have Raylib installed, let's learn the basics by building a chess game. We will progress **from simple to complex**, focusing on:

1. Drawing the chessboard
2. Loading and displaying chess pieces
3. Moving pieces with mouse interaction

Note: This tutorial focuses on graphics and interaction, not chess rules. We won't implement check, checkmate, or move validation.

Chapter 1: Basic Raylib Concepts

1.1 Colors in Raylib

Raylib provides predefined colors you can use directly:

Code block

```
1  RAYWHITE    // Light background color
2  WHITE       // Pure white
3  BLACK       // Pure black
4  RED         // Red
5  GREEN       // Green
6  BLUE        // Blue
7  YELLOW      // Yellow
8  ORANGE      // Orange
9  PURPLE      // Purple
10 GRAY        // Gray
11 DARKGRAY    // Dark gray
12 LIGHTGRAY   // Light gray
13 BROWN       // Brown
14 PINK        // Pink
```

You can also create custom colors:

Code block

```
1  Color myColor = {255, 100, 50, 255}; // R, G, B, Alpha (0-255 each)
2  Color semiTransparent = {0, 0, 0, 128}; // Black with 50% transparency
```

1.2 Drawing Basic Shapes

Code block

```
1  // Rectangle
2  DrawRectangle(x, y, width, height, color);
3
4  // Circle
5  DrawCircle(centerX, centerY, radius, color);
6
7  // Line
```

```
8 DrawLine(startX, startY, endX, endY, color);
9
10 // Text
11 DrawText("Hello", x, y, fontSize, color);
```

1.3 Input Handling

Code block

```
1 // Mouse position
2 Vector2 mousePos = GetMousePosition();
3 int mouseX = GetMouseX();
4 int mouseY = GetMouseY();
5
6 // Mouse button states
7 IsMouseButtonPressed(MOUSE_LEFT_BUTTON); // Just clicked
8 IsMouseButtonDown(MOUSE_LEFT_BUTTON); // Currently held down
9 IsMouseButtonReleased(MOUSE_LEFT_BUTTON); // Just released
10
11 // Keyboard
12 IsKeyPressed(KEY_A); // A key just pressed
13 IsKeyDown(KEY_SPACE); // Space bar held down
14 IsKeyReleased(KEY_ESCAPE); // ESC just released
```

1.4 Practical Example: Moving Ball with Sound

Let's create a complete example that combines shapes, keyboard input, and sound. This program displays a red ball that you can move with arrow keys. When the ball hits the screen border, it plays a sound.

Code block

```
1 #include "raylib.h"
2 #include <cmath> // For sinf()
3 #include <cstring> // For memset
4
5 int main() {
6     // Window setup
7     const int screenWidth = 800;
8     const int screenHeight = 600;
9     InitWindow(screenWidth, screenHeight, "Moving Ball with Sound");
10
11     // Initialize audio system
```

```

12     InitAudioDevice();
13
14     // Ball properties
15     float ballX = screenWidth / 2.0f;    // Start in center
16     float ballY = screenHeight / 2.0f;
17     const float ballRadius = 30.0f;
18     const float ballSpeed = 5.0f;        // Pixels per frame
19
20     // Create a simple "bounce" sound
21     // Method 1: Try to load from file (if bounce.wav exists)
22     Sound bounceSound = {0};
23     Wave bounceWave = {0};
24
25     // Check if bounce.wav file exists
26     if (FileExists("bounce.wav")) {
27         bounceWave = LoadWave("bounce.wav");
28         bounceSound = LoadSoundFromWave(bounceWave);
29     }
30
31     // Method 2: If file doesn't exist or failed to load, generate a simple
    tone
32     if (bounceSound.frameCount == 0) {
33         // Create a 440Hz tone for 0.2 seconds
34         Wave generatedWave = {0};
35         generatedWave.frameCount = static_cast<unsigned int>(44100 * 0.2f);
    // 0.2 seconds at 44100Hz
36         generatedWave.sampleRate = 44100;
37         generatedWave.sampleSize = 16;
38         generatedWave.channels = 1;
39
40         // Allocate memory for audio samples
41         size_t dataSize = generatedWave.frameCount * sizeof(short);
42         generatedWave.data = MemAlloc(dataSize);
43
44         // Clear memory first
45         if (generatedWave.data != nullptr) {
46             std::memset(generatedWave.data, 0, dataSize);
47
48             // Generate sine wave
49             short *data = static_cast<short*>(generatedWave.data);
50             for (unsigned int i = 0; i < generatedWave.frameCount; i++) {
51                 float time = static_cast<float>(i) / static_cast<float>
(generatedWave.sampleRate);
52                 float sineValue = std::sin(2.0f * 3.14159265f * 440.0f * time);
53                 data[i] = static_cast<short>(32000.0f * sineValue);
54             }
55

```

```

56         bounceSound = LoadSoundFromWave(generatedWave);
57         UnloadWave(generatedWave); // Can unload wave after creating sound
58     }
59 }
60
61 SetTargetFPS(60);
62
63 // Timer for displaying "BOUNCE!" text (in frames)
64 int bounceDisplayTimer = 0;
65 const int BOUNCE_DISPLAY_FRAMES = 45; // Show for 45 frames (~0.75
seconds at 60 FPS)
66
67 while (!WindowShouldClose()) {
68     // --- UPDATE ---
69
70     // Move ball with arrow keys
71     if (IsKeyDown(KEY_RIGHT)) ballX += ballSpeed;
72     if (IsKeyDown(KEY_LEFT)) ballX -= ballSpeed;
73     if (IsKeyDown(KEY_UP)) ballY -= ballSpeed;
74     if (IsKeyDown(KEY_DOWN)) ballY += ballSpeed;
75
76     // Check border collision and play sound
77     bool hitBorder = false;
78
79     // Left border
80     if (ballX - ballRadius < 0.0f) {
81         ballX = ballRadius; // Keep inside screen
82         hitBorder = true;
83     }
84     // Right border
85     if (ballX + ballRadius > static_cast<float>(screenWidth)) {
86         ballX = static_cast<float>(screenWidth) - ballRadius;
87         hitBorder = true;
88     }
89     // Top border
90     if (ballY - ballRadius < 0.0f) {
91         ballY = ballRadius;
92         hitBorder = true;
93     }
94     // Bottom border
95     if (ballY + ballRadius > static_cast<float>(screenHeight)) {
96         ballY = static_cast<float>(screenHeight) - ballRadius;
97         hitBorder = true;
98     }
99
100     // Play sound and start timer when hitting border
101     if (hitBorder && bounceSound.frameCount > 0) {

```

```

102         PlaySound(bounceSound);
103         bounceDisplayTimer = BOUNCE_DISPLAY_FRAMES; // Reset timer to
show text
104     }
105
106     // Decrease timer each frame (but not below 0)
107     if (bounceDisplayTimer > 0) {
108         bounceDisplayTimer--;
109     }
110
111     // --- DRAWING ---
112     BeginDrawing();
113
114     ClearBackground(RAYWHITE);
115
116     // Draw border lines to show boundaries
117     DrawRectangleLines(1, 1, screenWidth - 2, screenHeight - 2, DARKGRAY);
118
119     // Draw the ball
120     DrawCircle(static_cast<int>(ballX), static_cast<int>(ballY),
ballRadius, RED);
121
122     // Add a highlight to make it look 3D
123     Color highlightColor = {255, 100, 100, 200};
124     DrawCircle(static_cast<int>(ballX) - 8, static_cast<int>(ballY) - 8,
125         ballRadius / 3.0f, highlightColor);
126
127     // Draw instructions
128     DrawText("Use ARROW KEYS to move the ball", 10, 10, 20, DARKGRAY);
129     DrawText("Hit the border to hear a sound!", 10, 35, 20, DARKGRAY);
130     DrawText(TextFormat("Ball Position: (%.0f, %.0f)", ballX, ballY), 10,
60, 20, GRAY);
131
132     // Show "BOUNCE!" text when timer is active
133     if (bounceDisplayTimer > 0) {
134         // Calculate alpha (transparency) for fade effect
135         // Start at 255 (fully opaque) and fade to 0
136         unsigned char alpha = static_cast<unsigned char>(
137             255 * bounceDisplayTimer / BOUNCE_DISPLAY_FRAMES
138         );
139         Color bounceColor = {190, 33, 33, alpha}; // MAROON with variable
alpha
140
141         DrawText("BOUNCE!", screenWidth / 2 - 60, screenHeight / 2 - 20,
40, bounceColor);
142     }
143

```

```
144         EndDrawing();
145     }
146
147     // Cleanup
148     if (bounceSound.frameCount > 0) {
149         UnloadSound(bounceSound);
150     }
151     if (bounceWave.data != nullptr) {
152         UnloadWave(bounceWave);
153     }
154     CloseAudioDevice();
155     CloseWindow();
156
157     return 0;
158 }
```

Understanding the Code

Key Components:

Section	Purpose
<code>InitAudioDevice()</code>	Starts the audio system
<code>LoadSound()</code> / <code>LoadSoundFromWave()</code>	Loads sound effect
<code>PlaySound()</code>	Plays the loaded sound
<code>IsKeyDown()</code>	Checks if arrow keys are held
<code>ballX += ballSpeed</code>	Updates ball position
Border collision checks	Prevents ball from leaving screen
<code>bounceDisplayTimer</code>	Keeps "BOUNCE!" text visible for multiple frames
Fade effect	Text gradually fades out using alpha transparency
<code>TextFormat()</code>	Creates formatted text string
<code>UnloadSound()</code> / <code>CloseAudioDevice()</code>	Cleans up audio resources

Sound System Functions:

Code block

```
1 void InitAudioDevice(void);           // Initialize audio device
```

```

2  void CloseAudioDevice(void);           // Close audio device
3  bool IsAudioDeviceReady(void);         // Check if audio is ready
4
5  // Loading sounds
6  Sound LoadSound(const char *fileName); // Load from file (.wav, .ogg, .mp3)
7  Sound LoadSoundFromWave(Wave wave);    // Create from Wave data
8  void UnloadSound(Sound sound);         // Free sound memory
9
10 // Playing sounds
11 void PlaySound(Sound sound);            // Play once
12 void StopSound(Sound sound);           // Stop playing
13 void PauseSound(Sound sound);          // Pause
14 void ResumeSound(Sound sound);          // Resume
15 void SetSoundVolume(Sound sound, float volume); // 0.0 to 1.0
16 void SetSoundPitch(Sound sound, float pitch); // 1.0 = normal
17
18 // Wave operations
19 Wave LoadWave(const char *fileName);    // Load wave from file
20 void UnloadWave(Wave wave);             // Free wave data

```

How the Collision Detection Works:

1. Store the ball's position before moving
2. Update position based on key presses
3. Check if ball intersects with any screen border
4. If collision detected:
 - Clamp position to stay within bounds
 - Play the bounce sound
 - Reset the `bounceDisplayTimer` to show "BOUNCE!" text

Timer and Fade Effect:

Instead of showing "BOUNCE!" for just one frame (which is too fast to see), we use a timer system:

Code block

```

1  // When collision happens:
2  bounceDisplayTimer = 45; // Show text for 45 frames (~0.75 seconds)
3
4  // Each frame, decrease timer:
5  if (bounceDisplayTimer > 0) bounceDisplayTimer--;
6
7  // Draw text while timer is active:
8  if (bounceDisplayTimer > 0) {

```

```

9      // Calculate fade based on remaining time
10     unsigned char alpha = 255 * bounceDisplayTimer / 45;
11     DrawText("BOUNCE!", x, y, 40, {190, 33, 33, alpha});
12 }

```

The alpha value makes the text fade out gradually - it starts fully opaque (255) and becomes more transparent until it disappears.

Try Modifying:

- Change `ballSpeed` to make it move faster/slower
- Change `ballRadius` to make the ball bigger/smaller
- Change the color from `RED` to another color
- Add diagonal movement (check two keys at once)
- Add multiple balls
- Use `IsKeyPressed()` instead of `IsKeyDown()` for step-by-step movement

Chapter 2: Drawing the Chessboard

Let's start by creating a simple chessboard using rectangles.

Step 1: Basic Chessboard

Code block

```

1  #include "raylib.h"
2
3  int main() {
4      const int screenWidth = 800;
5      const int screenHeight = 800;
6      const int squareSize = 100; // Each square is 100x100 pixels
7
8      InitWindow(screenWidth, screenHeight, "Chess Board - Step 1");
9      SetTargetFPS(60);
10
11     while (!WindowShouldClose()) {
12         BeginDrawing();
13         ClearBackground(RAYWHITE);
14
15         // Draw 8x8 grid
16         for (int row = 0; row < 8; row++) {
17             for (int col = 0; col < 8; col++) {
18                 // Determine color: alternate between light and dark

```

```

19         Color squareColor;
20         if ((row + col) % 2 == 0) {
21             squareColor = {240, 217, 181, 255}; // Light wood color
22         } else {
23             squareColor = {181, 136, 99, 255}; // Dark wood color
24         }
25
26         // Calculate position
27         int x = col * squareSize;
28         int y = row * squareSize;
29
30         // Draw the square
31         DrawRectangle(x, y, squareSize, squareSize, squareColor);
32     }
33 }
34
35 EndDrawing();
36 }
37
38 CloseWindow();
39 return 0;
40 }

```

Understanding the Code

The Pattern Logic:

Code block

```
1  if ((row + col) % 2 == 0)
```

This creates the checkerboard pattern:

- When `row + col` is even → light color
- When `row + col` is odd → dark color

Example:

- Row 0, Col 0: $0+0=0$ (even) → Light
- Row 0, Col 1: $0+1=1$ (odd) → Dark
- Row 1, Col 0: $1+0=1$ (odd) → Dark
- Row 1, Col 1: $1+1=2$ (even) → Light

Step 2: Add Border and Labels

Let's enhance the board with coordinates (a-h, 1-8):

Code block

```
1  #include "raylib.h"
2
3  int main() {
4      const int squareSize = 100;
5      const int boardOffset = 50; // Space for labels
6      const int screenWidth = squareSize * 8 + boardOffset * 2;
7      const int screenHeight = squareSize * 8 + boardOffset * 2;
8
9      InitWindow(screenWidth, screenHeight, "Chess Board - Step 2");
10     SetTargetFPS(60);
11
12     // Colors
13     Color lightSquare = {240, 217, 181, 255};
14     Color darkSquare = {181, 136, 99, 255};
15     Color borderColor = {64, 64, 64, 255};
16
17     while (!WindowShouldClose()) {
18         BeginDrawing();
19         ClearBackground(RAYWHITE);
20
21         // Draw border background
22         DrawRectangle(0, 0, screenWidth, screenHeight, borderColor);
23
24         // Draw squares
25         for (int row = 0; row < 8; row++) {
26             for (int col = 0; col < 8; col++) {
27                 Color squareColor = ((row + col) % 2 == 0) ? lightSquare :
darkSquare;
28
29                 int x = boardOffset + col * squareSize;
30                 int y = boardOffset + row * squareSize;
31                 DrawRectangle(x, y, squareSize, squareSize, squareColor);
32             }
33         }
34
35         // Draw file labels (a-h) at the bottom
36         for (int col = 0; col < 8; col++) {
37             char label[2] = {'a' + col, '\0'};
38             int x = boardOffset + col * squareSize + squareSize/2 - 5;
39             int y = screenHeight - boardOffset + 15;
40             DrawText(label, x, y, 20, WHITE);
41         }
42
43         // Draw rank labels (1-8) on the left
```

```

43         for (int row = 0; row < 8; row++) {
44             char label[2] = {'8' - row, '\0'}; // 8 at top, 1 at bottom
45             int x = boardOffset - 30;
46             int y = boardOffset + row * squareSize + squareSize/2 - 10;
47             DrawText(label, x, y, 20, WHITE);
48         }
49
50         EndDrawing();
51     }
52
53     CloseWindow();
54     return 0;
55 }

```

Chapter 3: Chess Piece Representation

Before loading images, let's understand how to represent pieces. We'll use a simple approach with character codes.

3.1 Piece Representation

Code block

```

1  // Piece types (using single characters)
2  // Uppercase = White, Lowercase = Black
3  // K/k = King, Q/q = Queen, R/r = Rook, B/b = Bishop, N/n = Knight, P/p = Pawn
4
5  char board[8][8] = {
6      {'r', 'n', 'b', 'q', 'k', 'b', 'n', 'r'}, // Black back rank
7      {'p', 'p', 'p', 'p', 'p', 'p', 'p', 'p'}, // Black pawns
8      {' ', ' ', ' ', ' ', ' ', ' ', ' ', ' '}, // Empty
9      {' ', ' ', ' ', ' ', ' ', ' ', ' ', ' '}, // Empty
10     {' ', ' ', ' ', ' ', ' ', ' ', ' ', ' '}, // Empty
11     {' ', ' ', ' ', ' ', ' ', ' ', ' ', ' '}, // Empty
12     {'P', 'P', 'P', 'P', 'P', 'P', 'P', 'P'}, // White pawns
13     {'R', 'N', 'B', 'Q', 'K', 'B', 'N', 'R'} // White back rank
14 };

```

3.2 Loading Piece Images

For a polished look, we need piece images. You can download free chess piece images from:

- [Wikimedia Commons \(SVG chess pieces\)](#)

- [OpenGameArt.org](https://opengameart.org/)
- Or create your own simple images

Recommended: Use 60x60 or 64x64 PNG images with transparent backgrounds.

File naming convention:

Code block

```
1  pieces/
2  ├── w_king.png    // White king
3  ├── w_queen.png   // White queen
4  ├── w_rook.png    // White rook
5  ├── w_bishop.png  // White bishop
6  ├── w_knight.png  // White knight
7  ├── w_pawn.png    // White pawn
8  ├── b_king.png    // Black king
9  ├── b_queen.png   // Black queen
10 ├── b_rook.png    // Black rook
11 ├── b_bishop.png  // Black bishop
12 ├── b_knight.png  // Black knight
13 └── b_pawn.png    // Black pawn
```

Step 3: Drawing Pieces on the Board

Code block

```
1  #include "raylib.h"
2  #include <string>
3
4  // Convert piece character to filename
5  std::string GetPieceFilename(char piece) {
6      std::string prefix = (piece >= 'A' && piece <= 'Z') ? "w_" : "b_";
7      std::string name;
8
9      switch (tolower(piece)) {
10         case 'k': name = "king"; break;
11         case 'q': name = "queen"; break;
12         case 'r': name = "rook"; break;
13         case 'b': name = "bishop"; break;
14         case 'n': name = "knight"; break;
15         case 'p': name = "pawn"; break;
16         default: return "";
17     }
18
19     return "pieces/" + prefix + name + ".png";
20 }
```

```

21
22 int main() {
23     const int squareSize = 100;
24     const int boardOffset = 50;
25     const int screenWidth = squareSize * 8 + boardOffset * 2;
26     const int screenHeight = squareSize * 8 + boardOffset * 2;
27
28     InitWindow(screenWidth, screenHeight, "Chess Board - Step 3: Pieces");
29     SetTargetFPS(60);
30
31     // Initialize board
32     char board[8][8] = {
33         {'r', 'n', 'b', 'q', 'k', 'b', 'n', 'r'},
34         {'p', 'p', 'p', 'p', 'p', 'p', 'p', 'p'},
35         {' ', ' ', ' ', ' ', ' ', ' ', ' ', ' '},
36         {' ', ' ', ' ', ' ', ' ', ' ', ' ', ' '},
37         {' ', ' ', ' ', ' ', ' ', ' ', ' ', ' '},
38         {' ', ' ', ' ', ' ', ' ', ' ', ' ', ' '},
39         {'P', 'P', 'P', 'P', 'P', 'P', 'P', 'P'},
40         {'R', 'N', 'B', 'Q', 'K', 'B', 'N', 'R'}
41     };
42
43     // Load all piece textures
44     Texture2D pieceTextures[12]; // 6 piece types x 2 colors
45     std::string pieceFiles[] = {
46         "pieces/w_king.png", "pieces/w_queen.png", "pieces/w_rook.png",
47         "pieces/w_bishop.png", "pieces/w_knight.png", "pieces/w_pawn.png",
48         "pieces/b_king.png", "pieces/b_queen.png", "pieces/b_rook.png",
49         "pieces/b_bishop.png", "pieces/b_knight.png", "pieces/b_pawn.png"
50     };
51
52     for (int i = 0; i < 12; i++) {
53         pieceTextures[i] = LoadTexture(pieceFiles[i].c_str());
54     }
55
56     // Helper function to get texture index from piece character
57     auto GetTextureIndex = [](char piece) -> int {
58         bool isWhite = (piece >= 'A' && piece <= 'Z');
59         int baseIndex = isWhite ? 0 : 6;
60
61         switch (tolower(piece)) {
62             case 'k': return baseIndex + 0;
63             case 'q': return baseIndex + 1;
64             case 'r': return baseIndex + 2;
65             case 'b': return baseIndex + 3;
66             case 'n': return baseIndex + 4;
67             case 'p': return baseIndex + 5;

```

```

68         default: return -1;
69     }
70 };
71
72 while (!WindowShouldClose()) {
73     BeginDrawing();
74     ClearBackground(RAYWHITE);
75
76     // Draw border
77     DrawRectangle(0, 0, screenWidth, screenHeight, DARKGRAY);
78
79     // Draw squares
80     Color lightSquare = {240, 217, 181, 255};
81     Color darkSquare = {181, 136, 99, 255};
82
83     for (int row = 0; row < 8; row++) {
84         for (int col = 0; col < 8; col++) {
85             Color squareColor = ((row + col) % 2 == 0) ? lightSquare :
darkSquare;
86
87             int x = boardOffset + col * squareSize;
88             int y = boardOffset + row * squareSize;
89             DrawRectangle(x, y, squareSize, squareSize, squareColor);
90
91             // Draw piece if present
92             char piece = board[row][col];
93             if (piece != ' ') {
94                 int texIndex = GetTextureIndex(piece);
95                 if (texIndex >= 0) {
96                     // Center the piece in the square
97                     Texture2D tex = pieceTextures[texIndex];
98                     int pieceX = x + (squareSize - tex.width) / 2;
99                     int pieceY = y + (squareSize - tex.height) / 2;
100                     DrawTexture(tex, pieceX, pieceY, WHITE);
101                 }
102             }
103         }
104
105         // Draw labels (simplified)
106         for (int col = 0; col < 8; col++) {
107             char label[2] = {'a' + col, '\0'};
108             DrawText(label, boardOffset + col * squareSize + 40,
screenHeight - 35, 20, WHITE);
109         }
110
111         for (int row = 0; row < 8; row++) {
112             char label[2] = {'8' - row, '\0'};

```

```

113         DrawText(label, 15, boardOffset + row * squareSize + 35, 20,
        WHITE);
114     }
115
116     EndDrawing();
117 }
118
119 // Unload textures
120 for (int i = 0; i < 12; i++) {
121     UnloadTexture(pieceTextures[i]);
122 }
123
124 CloseWindow();
125 return 0;
126 }

```

Important: Place your piece images in a `pieces/` folder relative to your executable.

Chapter 4: Interactive Piece Movement

Now let's make the pieces movable with mouse interaction!

Key Concepts

1. **Selecting a piece:** Click on a square with a piece
2. **Dragging:** Move the piece with the mouse
3. **Dropping:** Release to place the piece on a new square
4. **Highlighting:** Show which square is selected

Step 4: Basic Piece Selection and Movement

Code block

```

1  #include "raylib.h"
2  #include <string>
3
4  // Board state
5  char board[8][8] = {
6      {'r', 'n', 'b', 'q', 'k', 'b', 'n', 'r'},
7      {'p', 'p', 'p', 'p', 'p', 'p', 'p', 'p'},
8      {' ', ' ', ' ', ' ', ' ', ' ', ' ', ' '},
9      {' ', ' ', ' ', ' ', ' ', ' ', ' ', ' '},
10     {' ', ' ', ' ', ' ', ' ', ' ', ' ', ' '},

```

```

11     {' ', ' ', ' ', ' ', ' ', ' ', ' ', ' '},
12     {'P', 'P', 'P', 'P', 'P', 'P', 'P', 'P'},
13     {'R', 'N', 'B', 'Q', 'K', 'B', 'N', 'R'}
14 };
15
16 // Game state
17 struct GameState {
18     int selectedRow = -1;    // -1 means no selection
19     int selectedCol = -1;
20     bool isDragging = false;
21     int dragPieceX = 0;      // Screen coordinates while dragging
22     int dragPieceY = 0;
23     char draggedPiece = ' '; // The piece being dragged
24 };
25
26 // Convert screen coordinates to board coordinates
27 bool ScreenToBoard(int screenX, int screenY, int boardOffset, int squareSize,
28                   int& outRow, int& outCol) {
29     int relativeX = screenX - boardOffset;
30     int relativeY = screenY - boardOffset;
31
32     // Check if within board bounds
33     if (relativeX < 0 || relativeX >= squareSize * 8) return false;
34     if (relativeY < 0 || relativeY >= squareSize * 8) return false;
35
36     outCol = relativeX / squareSize;
37     outRow = relativeY / squareSize;
38     return true;
39 }
40
41 int main() {
42     const int squareSize = 100;
43     const int boardOffset = 50;
44     const int screenWidth = squareSize * 8 + boardOffset * 2;
45     const int screenHeight = squareSize * 8 + boardOffset * 2;
46
47     InitWindow(screenWidth, screenHeight, "Chess - Step 4: Moving Pieces");
48     SetTargetFPS(60);
49
50     GameState state;
51
52     // Load textures (same as before)
53     Texture2D pieceTextures[12];
54     std::string pieceFiles[] = {
55         "pieces/w_king.png", "pieces/w_queen.png", "pieces/w_rook.png",
56         "pieces/w_bishop.png", "pieces/w_knight.png", "pieces/w_pawn.png",
57         "pieces/b_king.png", "pieces/b_queen.png", "pieces/b_rook.png",

```

```

58     "pieces/b_bishop.png", "pieces/b_knight.png", "pieces/b_pawn.png"
59 };
60 for (int i = 0; i < 12; i++) {
61     pieceTextures[i] = LoadTexture(pieceFiles[i].c_str());
62 }
63
64 auto GetTextureIndex = [](char piece) -> int {
65     bool isWhite = (piece >= 'A' && piece <= 'Z');
66     int baseIndex = isWhite ? 0 : 6;
67     switch (tolower(piece)) {
68         case 'k': return baseIndex + 0;
69         case 'q': return baseIndex + 1;
70         case 'r': return baseIndex + 2;
71         case 'b': return baseIndex + 3;
72         case 'n': return baseIndex + 4;
73         case 'p': return baseIndex + 5;
74         default: return -1;
75     }
76 };
77
78 while (!WindowShouldClose()) {
79     // --- INPUT HANDLING ---
80     Vector2 mousePos = GetMousePosition();
81     int mouseRow, mouseCol;
82     bool onBoard = ScreenToBoard(mousePos.x, mousePos.y, boardOffset,
83                                 squareSize, mouseRow, mouseCol);
84
85     // Mouse button pressed - try to select a piece
86     if (IsMouseButtonPressed(MOUSE_LEFT_BUTTON) && onBoard) {
87         char piece = board[mouseRow][mouseCol];
88         if (piece != ' ') {
89             state.isDragging = true;
90             state.draggedPiece = piece;
91             state.selectedRow = mouseRow;
92             state.selectedCol = mouseCol;
93             board[mouseRow][mouseCol] = ' '; // Remove from board
94             temporarily
95         }
96     }
97
98     // Mouse button released - drop the piece
99     if (IsMouseButtonReleased(MOUSE_LEFT_BUTTON) && state.isDragging) {
100         if (onBoard) {
101             // Place piece at new position
102             board[mouseRow][mouseCol] = state.draggedPiece;
103         } else {
104             // Drop off board - return to original position

```

```

104         board[state.selectedRow][state.selectedCol] =
state.draggedPiece;
105     }
106
107     // Reset drag state
108     state.isDragging = false;
109     state.draggedPiece = ' ';
110     state.selectedRow = -1;
111     state.selectedCol = -1;
112 }
113
114 // Update drag position
115 if (state.isDragging) {
116     state.dragPieceX = mousePos.x - squareSize / 2;
117     state.dragPieceY = mousePos.y - squareSize / 2;
118 }
119
120 // --- DRAWING ---
121 BeginDrawing();
122 ClearBackground(RAYWHITE);
123
124 // Draw border
125 DrawRectangle(0, 0, screenWidth, screenHeight, DARKGRAY);
126
127 // Draw squares
128 Color lightSquare = {240, 217, 181, 255};
129 Color darkSquare = {181, 136, 99, 255};
130 Color highlightColor = {255, 255, 0, 100}; // Yellow semi-transparent
131
132 for (int row = 0; row < 8; row++) {
133     for (int col = 0; col < 8; col++) {
134         int x = boardOffset + col * squareSize;
135         int y = boardOffset + row * squareSize;
136
137         // Draw square
138         Color squareColor = ((row + col) % 2 == 0) ? lightSquare :
darkSquare;
139         DrawRectangle(x, y, squareSize, squareSize, squareColor);
140
141         // Highlight selected square
142         if (row == state.selectedRow && col == state.selectedCol) {
143             DrawRectangle(x, y, squareSize, squareSize,
highlightColor);
144         }
145
146         // Draw piece (only if not being dragged)
147         char piece = board[row][col];

```

```

148         if (piece != ' ') {
149             int texIndex = GetTextureIndex(piece);
150             if (texIndex >= 0) {
151                 Texture2D tex = pieceTextures[texIndex];
152                 int pieceX = x + (squareSize - tex.width) / 2;
153                 int pieceY = y + (squareSize - tex.height) / 2;
154                 DrawTexture(tex, pieceX, pieceY, WHITE);
155             }
156         }
157     }
158 }
159
160 // Draw dragged piece at mouse position
161 if (state.isDragging) {
162     int texIndex = GetTextureIndex(state.draggedPiece);
163     if (texIndex >= 0) {
164         DrawTexture(pieceTextures[texIndex], state.dragPieceX,
165                     state.dragPieceY, WHITE);
166     }
167 }
168
169 // Draw instructions
170 DrawText("Click and drag pieces to move them", 10, 10, 20, DARKGRAY);
171
172 EndDrawing();
173 }
174
175 // Cleanup
176 for (int i = 0; i < 12; i++) {
177     UnloadTexture(pieceTextures[i]);
178 }
179 CloseWindow();
180 return 0;
181 }

```

Understanding the Movement Logic

1. Selection Phase:

Code block

```

1  if (IsMouseButtonPressed(MOUSE_LEFT_BUTTON) && onBoard) {
2      char piece = board[mouseRow][mouseCol];
3      if (piece != ' ') {
4          state.isDragging = true;
5          state.draggedPiece = piece;
6          board[mouseRow][mouseCol] = ' '; // Temporarily remove from board

```

```
7     }  
8 }
```

2. Dragging Phase:

- Piece follows mouse cursor
- Original square is highlighted

3. Drop Phase:

Code block

```
1  if (IsMouseButtonReleased(MOUSE_LEFT_BUTTON) && state.isDragging) {  
2      if (onBoard) {  
3          board[mouseRow][mouseCol] = state.draggedPiece; // Place at new  
        position  
4      } else {  
5          board[state.selectedRow][state.selectedCol] = state.draggedPiece; //  
        Return  
6      }  
7      state.isDragging = false;  
8  }
```

Chapter 5: Visual Enhancements

Let's add visual polish to make the game look better.

Step 5: Highlighting Valid Moves and Hover Effects

Code block

```
1  #include "raylib.h"  
2  #include <string>  
3  #include <cmath>  
4  
5  char board[8][8] = {  
6      {'r', 'n', 'b', 'q', 'k', 'b', 'n', 'r'},  
7      {'p', 'p', 'p', 'p', 'p', 'p', 'p', 'p'},  
8      {' ', ' ', ' ', ' ', ' ', ' ', ' ', ' '},  
9      {' ', ' ', ' ', ' ', ' ', ' ', ' ', ' '},  
10     {' ', ' ', ' ', ' ', ' ', ' ', ' ', ' '},  
11     {' ', ' ', ' ', ' ', ' ', ' ', ' ', ' '},  
12     {'P', 'P', 'P', 'P', 'P', 'P', 'P', 'P'},  
13     {'R', 'N', 'B', 'Q', 'K', 'B', 'N', 'R'}
```

```

14 };
15
16 struct GameState {
17     int selectedRow = -1;
18     int selectedCol = -1;
19     bool isDragging = false;
20     int dragPieceX = 0;
21     int dragPieceY = 0;
22     char draggedPiece = ' ';
23     int hoverRow = -1;    // Current square under mouse
24     int hoverCol = -1;
25 };
26
27 bool ScreenToBoard(int screenX, int screenY, int boardOffset, int squareSize,
28                   int& outRow, int& outCol) {
29     int relativeX = screenX - boardOffset;
30     int relativeY = screenY - boardOffset;
31     if (relativeX < 0 || relativeX >= squareSize * 8) return false;
32     if (relativeY < 0 || relativeY >= squareSize * 8) return false;
33     outCol = relativeX / squareSize;
34     outRow = relativeY / squareSize;
35     return true;
36 }
37
38 // Draw a rounded rectangle (using multiple rectangles and circles)
39 void DrawRoundedRect(int x, int y, int width, int height, int radius, Color
40 color) {
41     DrawRectangle(x + radius, y, width - 2*radius, height, color);
42     DrawRectangle(x, y + radius, width, height - 2*radius, color);
43     DrawCircle(x + radius, y + radius, radius, color);
44     DrawCircle(x + width - radius, y + radius, radius, color);
45     DrawCircle(x + radius, y + height - radius, radius, color);
46     DrawCircle(x + width - radius, y + height - radius, radius, color);
47 }
48
49 int main() {
50     const int squareSize = 100;
51     const int boardOffset = 60;
52     const int screenWidth = squareSize * 8 + boardOffset * 2;
53     const int screenHeight = squareSize * 8 + boardOffset * 2;
54
55     InitWindow(screenWidth, screenHeight, "Chess - Step 5: Visual Polish");
56     SetTargetFPS(60);
57
58     GameState state;
59
60     // Colors

```

```

60     Color lightSquare = {240, 217, 181, 255};
61     Color darkSquare = {181, 136, 99, 255};
62     Color borderColor = {48, 46, 43, 255};
63     Color hoverColor = {255, 255, 255, 40};           // Subtle white glow
64     Color selectColor = {255, 255, 0, 60};           // Yellow highlight
65     Color validMoveColor = {0, 255, 0, 40};          // Green for valid moves
66
67     // Load textures
68     Texture2D pieceTextures[12];
69     std::string pieceFiles[] = {
70         "pieces/w_king.png", "pieces/w_queen.png", "pieces/w_rook.png",
71         "pieces/w_bishop.png", "pieces/w_knight.png", "pieces/w_pawn.png",
72         "pieces/b_king.png", "pieces/b_queen.png", "pieces/b_rook.png",
73         "pieces/b_bishop.png", "pieces/b_knight.png", "pieces/b_pawn.png"
74     };
75     for (int i = 0; i < 12; i++) {
76         pieceTextures[i] = LoadTexture(pieceFiles[i].c_str());
77     }
78
79     auto GetTextureIndex = [](char piece) -> int {
80         bool isWhite = (piece >= 'A' && piece <= 'Z');
81         int baseIndex = isWhite ? 0 : 6;
82         switch (tolower(piece)) {
83             case 'k': return baseIndex + 0;
84             case 'q': return baseIndex + 1;
85             case 'r': return baseIndex + 2;
86             case 'b': return baseIndex + 3;
87             case 'n': return baseIndex + 4;
88             case 'p': return baseIndex + 5;
89             default: return -1;
90         }
91     };
92
93     while (!WindowShouldClose()) {
94         // --- INPUT HANDLING ---
95         Vector2 mousePos = GetMousePosition();
96         int mouseRow, mouseCol;
97         bool onBoard = ScreenToBoard(mousePos.x, mousePos.y, boardOffset,
98                                     squareSize, mouseRow, mouseCol);
99
100        // Update hover state
101        if (onBoard) {
102            state.hoverRow = mouseRow;
103            state.hoverCol = mouseCol;
104        } else {
105            state.hoverRow = -1;
106            state.hoverCol = -1;

```

```

107     }
108
109     // Mouse button pressed
110     if (IsMouseButtonPressed(MOUSE_LEFT_BUTTON) && onBoard) {
111         char piece = board[mouseRow][mouseCol];
112         if (piece != ' ') {
113             state.isDragging = true;
114             state.draggedPiece = piece;
115             state.selectedRow = mouseRow;
116             state.selectedCol = mouseCol;
117             board[mouseRow][mouseCol] = ' ';
118         }
119     }
120
121     // Mouse button released
122     if (IsMouseButtonReleased(MOUSE_LEFT_BUTTON) && state.isDragging) {
123         if (onBoard) {
124             board[mouseRow][mouseCol] = state.draggedPiece;
125         } else {
126             board[state.selectedRow][state.selectedCol] =
state.draggedPiece;
127         }
128         state.isDragging = false;
129         state.draggedPiece = ' ';
130         state.selectedRow = -1;
131         state.selectedCol = -1;
132     }
133
134     if (state.isDragging) {
135         state.dragPieceX = mousePos.x - squareSize / 2;
136         state.dragPieceY = mousePos.y - squareSize / 2;
137     }
138
139     // --- DRAWING ---
140     BeginDrawing();
141     ClearBackground(RAYWHITE);
142
143     // Draw outer border with rounded corners
144     DrawRectangle(0, 0, screenWidth, screenHeight, borderColor);
145
146     // Draw inner board background
147     int boardSize = squareSize * 8;
148     DrawRectangle(boardOffset - 5, boardOffset - 5,
boardSize + 10, boardSize + 10, borderColor);
149
150
151     // Draw squares with highlights
152     for (int row = 0; row < 8; row++) {

```

```

153         for (int col = 0; col < 8; col++) {
154             int x = boardOffset + col * squareSize;
155             int y = boardOffset + row * squareSize;
156
157             // Base square color
158             Color squareColor = ((row + col) % 2 == 0) ? lightSquare :
darkSquare;
159             DrawRectangle(x, y, squareSize, squareSize, squareColor);
160
161             // Hover effect
162             if (row == state.hoverRow && col == state.hoverCol &&
!state.isDragging) {
163                 DrawRectangle(x, y, squareSize, squareSize, hoverColor);
164             }
165
166             // Selected square highlight
167             if (row == state.selectedRow && col == state.selectedCol) {
168                 DrawRectangle(x, y, squareSize, squareSize, selectColor);
169                 // Draw border around selected square
170                 DrawRectangleLines(x, y, squareSize, squareSize, YELLOW);
171             }
172
173             // Draw piece
174             char piece = board[row][col];
175             if (piece != ' ') {
176                 int texIndex = GetTextureIndex(piece);
177                 if (texIndex >= 0) {
178                     Texture2D tex = pieceTextures[texIndex];
179                     int pieceX = x + (squareSize - tex.width) / 2;
180                     int pieceY = y + (squareSize - tex.height) / 2;
181                     DrawTexture(tex, pieceX, pieceY, WHITE);
182                 }
183             }
184         }
185     }
186
187     // Draw dragged piece (larger and centered on mouse)
188     if (state.isDragging) {
189         int texIndex = GetTextureIndex(state.draggedPiece);
190         if (texIndex >= 0) {
191             Texture2D tex = pieceTextures[texIndex];
192             // Scale up slightly while dragging
193             float scale = 1.1f;
194             int scaledWidth = tex.width * scale;
195             int scaledHeight = tex.height * scale;
196             int drawX = mousePos.x - scaledWidth / 2;
197             int drawY = mousePos.y - scaledHeight / 2;

```

```

198
199         // Draw shadow
200         DrawRectangle(drawX + 5, drawY + 5, scaledWidth, scaledHeight,
201             {0, 0, 0, 100});
202         // Draw scaled piece
203         DrawTextureEx(tex, {float(drawX), float(drawY)}, 0, scale,
WHITE);
204     }
205 }
206
207     // Draw coordinates
208     for (int col = 0; col < 8; col++) {
209         char fileLabel[2] = {'a' + col, '\0'};
210         DrawText(fileLabel, boardOffset + col * squareSize + squareSize/2
- 5,
211             boardSize + boardOffset + 10, 20, WHITE);
212     }
213     for (int row = 0; row < 8; row++) {
214         char rankLabel[2] = {'8' - row, '\0'};
215         DrawText(rankLabel, boardOffset - 30,
216             boardOffset + row * squareSize + squareSize/2 - 10, 20,
WHITE);
217     }
218
219     // Draw instructions
220     DrawText("Hover to preview, click and drag to move pieces",
221         boardOffset, 15, 20, WHITE);
222
223     EndDrawing();
224 }
225
226     for (int i = 0; i < 12; i++) {
227         UnloadTexture(pieceTextures[i]);
228     }
229     CloseWindow();
230     return 0;
231 }

```

Chapter 6: Complete Chess Board Application

Here's the final, complete code with all features:

Features Summary

- Chessboard with proper colors
- All pieces in starting positions
- Click and drag to move pieces
- Hover highlighting
- Selected square highlighting
- Visual polish (shadows, borders)
- Coordinates display

Code block

```

1  #include "raylib.h"
2  #include <string>
3
4  // =====
5  // CONSTANTS AND CONFIGURATION
6  // =====
7  const int SQUARE_SIZE = 100;
8  const int BOARD_OFFSET = 60;
9  const int SCREEN_WIDTH = SQUARE_SIZE * 8 + BOARD_OFFSET * 2;
10 const int SCREEN_HEIGHT = SQUARE_SIZE * 8 + BOARD_OFFSET * 2;
11
12 // Colors
13 const Color COLOR_LIGHT_SQUARE = {240, 217, 181, 255};
14 const Color COLOR_DARK_SQUARE = {181, 136, 99, 255};
15 const Color COLOR_BORDER = {48, 46, 43, 255};
16 const Color COLOR_HOVER = {255, 255, 255, 40};
17 const Color COLOR_SELECT = {255, 255, 0, 60};
18
19 // =====
20 // GAME STATE
21 // =====
22 struct ChessGame {
23     char board[8][8];
24     int selectedRow = -1;
25     int selectedCol = -1;
26     bool isDragging = false;
27     Vector2 dragPosition = {0, 0};
28     char draggedPiece = ' ';
29     int hoverRow = -1;
30     int hoverCol = -1;
31     Texture2D pieceTextures[12];
32
33     void InitializeBoard() {
34         char initial[8][8] = {

```

```

35         {'r', 'n', 'b', 'q', 'k', 'b', 'n', 'r'},
36         {'p', 'p', 'p', 'p', 'p', 'p', 'p', 'p'},
37         {' ', ' ', ' ', ' ', ' ', ' ', ' ', ' '},
38         {' ', ' ', ' ', ' ', ' ', ' ', ' ', ' '},
39         {' ', ' ', ' ', ' ', ' ', ' ', ' ', ' '},
40         {' ', ' ', ' ', ' ', ' ', ' ', ' ', ' '},
41         {'P', 'P', 'P', 'P', 'P', 'P', 'P', 'P'},
42         {'R', 'N', 'B', 'Q', 'K', 'B', 'N', 'R'}
43     };
44     for (int r = 0; r < 8; r++)
45         for (int c = 0; c < 8; c++)
46             board[r][c] = initial[r][c];
47 }
48
49 void LoadTextures() {
50     const char* files[] = {
51         "pieces/w_king.png", "pieces/w_queen.png", "pieces/w_rook.png",
52         "pieces/w_bishop.png", "pieces/w_knight.png", "pieces/w_pawn.png",
53         "pieces/b_king.png", "pieces/b_queen.png", "pieces/b_rook.png",
54         "pieces/b_bishop.png", "pieces/b_knight.png", "pieces/b_pawn.png"
55     };
56     for (int i = 0; i < 12; i++) {
57         pieceTextures[i] = LoadTexture(files[i]);
58     }
59 }
60
61 void UnloadTextures() {
62     for (int i = 0; i < 12; i++) {
63         UnloadTexture(pieceTextures[i]);
64     }
65 }
66
67 int GetTextureIndex(char piece) {
68     bool isWhite = (piece >= 'A' && piece <= 'Z');
69     int base = isWhite ? 0 : 6;
70     switch (tolower(piece)) {
71         case 'k': return base + 0;
72         case 'q': return base + 1;
73         case 'r': return base + 2;
74         case 'b': return base + 3;
75         case 'n': return base + 4;
76         case 'p': return base + 5;
77         default: return -1;
78     }
79 }
80 };
81

```

```

82 // =====
83 // UTILITY FUNCTIONS
84 // =====
85 bool ScreenToBoard(int screenX, int screenY, int& row, int& col) {
86     int relX = screenX - BOARD_OFFSET;
87     int relY = screenY - BOARD_OFFSET;
88     if (relX < 0 || relX >= SQUARE_SIZE * 8) return false;
89     if (relY < 0 || relY >= SQUARE_SIZE * 8) return false;
90     col = relX / SQUARE_SIZE;
91     row = relY / SQUARE_SIZE;
92     return true;
93 }
94
95 // =====
96 // INPUT HANDLING
97 // =====
98 void HandleInput(ChessGame& game) {
99     Vector2 mousePos = GetMousePosition();
100     int row, col;
101     bool onBoard = ScreenToBoard(mousePos.x, mousePos.y, row, col);
102
103     // Update hover
104     if (onBoard) {
105         game.hoverRow = row;
106         game.hoverCol = col;
107     } else {
108         game.hoverRow = -1;
109         game.hoverCol = -1;
110     }
111
112     // Mouse pressed - select piece
113     if (IsMouseButtonPressed(MOUSE_LEFT_BUTTON) && onBoard) {
114         char piece = game.board[row][col];
115         if (piece != ' ') {
116             game.isDragging = true;
117             game.draggedPiece = piece;
118             game.selectedRow = row;
119             game.selectedCol = col;
120             game.board[row][col] = ' ';
121         }
122     }
123
124     // Mouse released - drop piece
125     if (IsMouseButtonReleased(MOUSE_LEFT_BUTTON) && game.isDragging) {
126         if (onBoard) {
127             game.board[row][col] = game.draggedPiece;
128         } else {

```

```

129         game.board[game.selectedRow][game.selectedCol] = game.draggedPiece;
130     }
131     game.isDragging = false;
132     game.draggedPiece = ' ';
133     game.selectedRow = -1;
134     game.selectedCol = -1;
135 }
136
137 // Update drag position
138 if (game.isDragging) {
139     game.dragPosition = {mousePos.x - SQUARE_SIZE/2.0f,
140                          mousePos.y - SQUARE_SIZE/2.0f};
141 }
142 }
143
144 // =====
145 // RENDERING
146 // =====
147 void DrawChessBoard(ChessGame& game) {
148     // Border
149     DrawRectangle(0, 0, SCREEN_WIDTH, SCREEN_HEIGHT, COLOR_BORDER);
150
151     // Inner board edge
152     int boardSize = SQUARE_SIZE * 8;
153     DrawRectangle(BOARD_OFFSET - 5, BOARD_OFFSET - 5,
154                  boardSize + 10, boardSize + 10, COLOR_BORDER);
155
156     // Squares
157     for (int row = 0; row < 8; row++) {
158         for (int col = 0; col < 8; col++) {
159             int x = BOARD_OFFSET + col * SQUARE_SIZE;
160             int y = BOARD_OFFSET + row * SQUARE_SIZE;
161
162             // Base color
163             Color squareColor = ((row + col) % 2 == 0) ?
164                                COLOR_LIGHT_SQUARE : COLOR_DARK_SQUARE;
165             DrawRectangle(x, y, SQUARE_SIZE, SQUARE_SIZE, squareColor);
166
167             // Hover effect
168             if (row == game.hoverRow && col == game.hoverCol &&
169 !game.isDragging) {
170                 DrawRectangle(x, y, SQUARE_SIZE, SQUARE_SIZE, COLOR_HOVER);
171             }
172
173             // Selection highlight
174             if (row == game.selectedRow && col == game.selectedCol) {
175                 DrawRectangle(x, y, SQUARE_SIZE, SQUARE_SIZE, COLOR_SELECT);
176             }
177         }
178     }
179 }

```

```

175         DrawRectangleLines(x, y, SQUARE_SIZE, SQUARE_SIZE, YELLOW);
176     }
177
178     // Piece
179     char piece = game.board[row][col];
180     if (piece != ' ') {
181         int idx = game.GetTextureIndex(piece);
182         if (idx >= 0) {
183             Texture2D tex = game.pieceTextures[idx];
184             int px = x + (SQUARE_SIZE - tex.width) / 2;
185             int py = y + (SQUARE_SIZE - tex.height) / 2;
186             DrawTexture(tex, px, py, WHITE);
187         }
188     }
189 }
190 }
191 }
192
193 void DrawDraggedPiece(ChessGame& game) {
194     if (!game.isDragging) return;
195
196     int idx = game.GetTextureIndex(game.draggedPiece);
197     if (idx < 0) return;
198
199     Texture2D tex = game.pieceTextures[idx];
200     float scale = 1.1f;
201     int scaledW = tex.width * scale;
202     int scaledH = tex.height * scale;
203     int drawX = game.dragPosition.x + SQUARE_SIZE/2 - scaledW/2;
204     int drawY = game.dragPosition.y + SQUARE_SIZE/2 - scaledH/2;
205
206     // Shadow
207     DrawRectangle(drawX + 5, drawY + 5, scaledW, scaledH, {0, 0, 0, 100});
208     // Scaled piece
209     DrawTextureEx(tex, {float(drawX), float(drawY)}, 0, scale, WHITE);
210 }
211
212 void DrawCoordinates() {
213     int boardSize = SQUARE_SIZE * 8;
214
215     // File labels (a-h)
216     for (int col = 0; col < 8; col++) {
217         char label[2] = {'a' + col, '\0'};
218         DrawText(label, BOARD_OFFSET + col * SQUARE_SIZE + SQUARE_SIZE/2 - 5,
219             BOARD_OFFSET + boardSize + 10, 20, WHITE);
220     }
221

```

```

222     // Rank labels (1-8)
223     for (int row = 0; row < 8; row++) {
224         char label[2] = {'8' - row, '\0'};
225         DrawText(label, BOARD_OFFSET - 30,
226                 BOARD_OFFSET + row * SQUARE_SIZE + SQUARE_SIZE/2 - 10, 20,
227         WHITE);
228     }
229
230     // =====
231     // MAIN
232     // =====
233     int main() {
234         InitWindow(SCREEN_WIDTH, SCREEN_HEIGHT, "Complete Chess Board
235         Application");
236         SetTargetFPS(60);
237
238         ChessGame game;
239         game.InitializeBoard();
240         game.LoadTextures();
241
242         while (!WindowShouldClose()) {
243             HandleInput(game);
244
245             BeginDrawing();
246             ClearBackground(RAYWHITE);
247
248             DrawChessBoard(game);
249             DrawDraggedPiece(game);
250             DrawCoordinates();
251
252             // Instructions
253             DrawText("Click and drag pieces to move them",
254                     BOARD_OFFSET, 15, 20, WHITE);
255             DrawText("Press ESC to exit", BOARD_OFFSET, SCREEN_HEIGHT - 35, 18,
256             LIGHTGRAY);
257
258             EndDrawing();
259         }
260
261         game.UnloadTextures();
262         CloseWindow();
263         return 0;
264     }

```

Chapter 7: Exercises for Further Learning

Now that you have a working chess board, try these exercises:

Exercise 1: Reset Button

Add a button to reset the board to starting position.

Hint: Use `DrawRectangle()` and `CheckCollisionPointRec()` for button detection.

Exercise 2: Sound Effects

Add sounds when picking up and dropping pieces.

Hint: Use `LoadSound()`, `PlaySound()`, `UnloadSound()`.

Exercise 3: Last Move Highlight

Highlight the squares of the last move (from -> to).

Hint: Store the last move in your game state.

Exercise 4: Flip Board

Add a key press (like 'F') to flip the board perspective.

Hint: Change how you calculate `row` when drawing.

Exercise 5: Simple Move Validation

Only allow moves to empty squares (no captures yet).

Hint: Check the destination square before placing the piece.

Exercise 6: Save/Load Position

Save the current board state to a file and load it later.

Hint: Write the board array to a text file.

Summary of Raylib Functions Used

Category	Functions
Window	<code>InitWindow()</code> , <code>CloseWindow()</code> , <code>WindowShouldClose()</code> , <code>SetTargetFPS()</code>
Drawing	<code>BeginDrawing()</code> , <code>EndDrawing()</code> , <code>ClearBackground()</code>

Shapes	<code>DrawRectangle()</code> , <code>DrawRectangleLines()</code> , <code>DrawCircle()</code> , <code>DrawRectangleRec()</code>
Text	<code>DrawText()</code>
Textures	<code>LoadTexture()</code> , <code>UnloadTexture()</code> , <code>DrawTexture()</code> , <code>DrawTextureEx()</code>
Input	<code>GetMousePosition()</code> , <code>GetMouseX()</code> , <code>GetMouseY()</code> , <code>IsMouseButtonPressed()</code> , <code>IsMouseButtonReleased()</code> , <code>IsKeyPressed()</code>
Collision	<code>CheckCollisionPointRec()</code>

Conclusion

Raylib is an excellent library for learning game programming. Start with simple shapes and text, then gradually add more complexity. Remember:

- **Read the examples:** They are your best learning resource
- **Experiment:** Change values and see what happens
- **Ask for help:** The community is friendly and supportive
- **Have fun:** Game programming should be enjoyable!

Happy coding!

This guide was created for C++ beginners. If you find any errors or have suggestions for improvement, please share your feedback.

